



ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

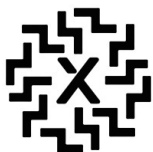
## Semester Project Report: W25Q128JW Controller for X-HEEP

DESIGN, VERIFICATION, AND INTEGRATION OF SPI  
ROUTINE AND HW FUNCTIONS FOR EFFICIENT  
MASTER-TO-SLAVE OFF-CHIP COMMUNICATIONS

---

Thomas Lenges under supervision of Dr. Pasquale Davide Schiavone and PhD  
Candidate Tommaso Terzano

January 13, 2026



# Contents

|          |                         |           |
|----------|-------------------------|-----------|
| <b>1</b> | <b>Introduction</b>     | <b>2</b>  |
| 1.1      | Goals . . . . .         | 2         |
| 1.2      | Context . . . . .       | 2         |
| 1.3      | W25Q128JW . . . . .     | 2         |
| <b>2</b> | <b>Hardware</b>         | <b>3</b>  |
| 2.1      | READ FSM . . . . .      | 6         |
| 2.2      | DMA_INIT FSM . . . . .  | 7         |
| 2.3      | FWAIT FSM . . . . .     | 8         |
| 2.4      | ERASE FSM . . . . .     | 9         |
| 2.5      | MODIFY FSM . . . . .    | 9         |
| 2.6      | WRITE FSM . . . . .     | 10        |
| <b>3</b> | <b>Software</b>         | <b>12</b> |
| 3.1      | Driver . . . . .        | 12        |
| 3.1.1    | Operation . . . . .     | 12        |
| 3.1.2    | Polling . . . . .       | 12        |
| 3.1.3    | Interrupt . . . . .     | 12        |
| 3.2      | Applications . . . . .  | 12        |
| 3.2.1    | Read example . . . . .  | 13        |
| 3.2.2    | Write example . . . . . | 13        |
| <b>4</b> | <b>Testing</b>          | <b>13</b> |
| 4.1      | Write example . . . . . | 14        |
| <b>5</b> | <b>Conclusion</b>       | <b>16</b> |

# 1 Introduction

## 1.1 Goals

The general limited size of MicroController Units' (MCU) internal Random Access Memory (RAM) necessitates the use of external memory. However, direct access to external memory through the Central Processing Unit (CPU) is a slow process and significantly increases CPU workload. This requires configuring an external memory controller (Serial Peripheral Interface (SPI) in our case), which is managed by the CPU. To reduce CPU involvement and improve data transfer efficiency, Direct Memory Access (DMA) is employed to handle data movement independently of the CPU.

Hence, the project seeks to improve overall system performance and scalability by offloading repetitive and time-critical operations from the CPU to dedicated hardware (HW) modules implemented in SystemVerilog. This approach enables parallel execution, reduces latency, and lowers power consumption.

To achieve this goal, a new HW module with interrupt-driven operation is implemented. This module is configured by the CPU through software (SW) and autonomously sets up both the SPI controller [1] and the DMA engine [2]. It is integrated into X-HEEP [2] as a new peripheral and is referred to as the *W25Q128JW controller*.

## 1.2 Context

Project begins with SW<sup>1</sup> that performs read and write operations. This software serves as the reference and inspiration for designing the corresponding HW module. The SW is executed by the CPU, which configures the SPI flash controller to communicate with the external W25Q128JW flash memory [3]. In addition, the SW interfaces with the DMA to manage data transfers, including SPI-to-RAM, RAM-to-SPI, and RAM-to-RAM operations.

## 1.3 W25Q128JW

The W25Q128JW is a serial NOR flash memory device with a total storage capacity of 128 Mbits (16 MBytes). The memory array is organized into 256 blocks, each block consisting of 16 sectors. Each sector has a size of 4 KBytes and is further divided into 16 pages of 256 bytes, which represent the minimum programmable unit of the memory.

A read operation is performed as follows. The W25Q128JW receives two commands over the SPI interface. The first command is a word that specifies the address within the flash memory and the operation type ("read" in this case). The second command is a word with specifications for the previous command, which defines the transfer parameters, including the amount of data to be read, the data direction (from the flash memory to the SPI interface), and the transfer mode (e.g., single SPI or quad SPI). The read operation supports byte-level granularity.

A write operation is more intricate due to the following technology limitation: a cell containing a memory information of logic '0' cannot be reprogrammed to '1' with cell precision. The smallest unit which supports such operation is a sector (4 KBytes). Hence in order to complete a write operation, the following must be achieved: the sector in which one wants to write must first be read and stored within a RAM buffer. Then, the concerned sector must be erased (set all its unit cells to logic '1'). The RAM buffer is then internally modified with the "write data" and is finally written back to the flash memory. This final step is done page (256 bytes) by page (as this is the smallest programmable unit) until the entire sector is written back. The write

---

<sup>1</sup>located in sw/device/bsp/w25q and sw/application/example\_spi\_read/write

operation supports word-level granularity and is repeated sector after sector until the entire data has been written into the flash memory.

## 2 Hardware

The top-level logic architecture of the *W25Q128JW controller* is illustrated in Figure 1, while the hardware architecture and interconnections of the controller are shown in Figure 3.

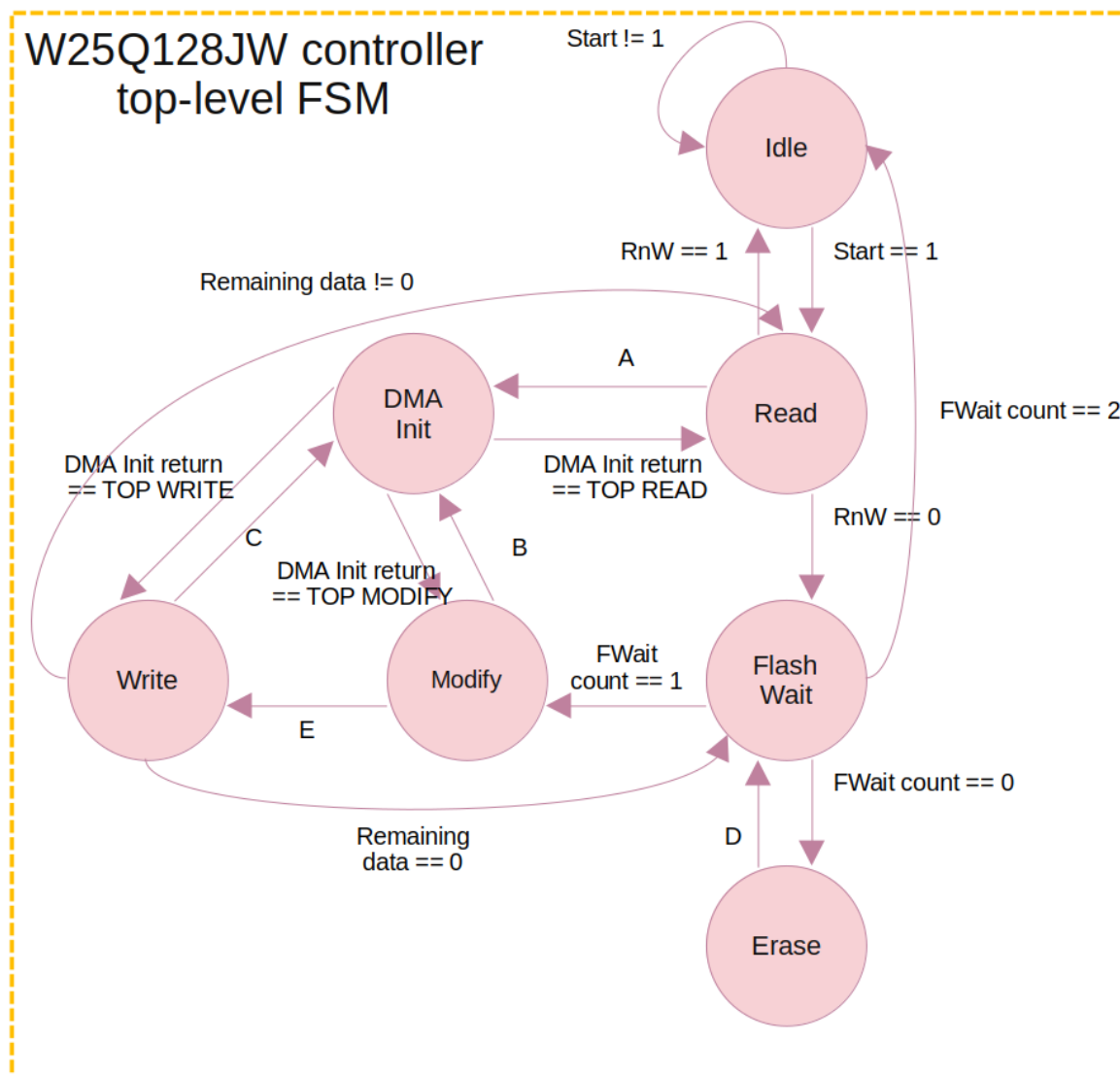


Figure 1: *W25Q128JW* controller top-level FSM architecture. Initial state is *Idle*.

The FSM shown is simplified for resume purposes. While the module does not receive a start signal, the FSM remains in the *Idle* state. Once this signal is received, the FSM transitions to the *READ* sub-FSM, which immediately enters the *DMA Init* sub-FSM. This sub-FSM is invoked whenever a DMA operation is required and returns control to its caller upon completion, as illustrated by paths A, B, and C. Once the *READ* sub-FSM is complete (explained in further details in its corresponding subsection below), the FSM either returns to the *Idle* state if the operation is a simple read from the W25Q128JW flash or proceeds to the *Flash Wait* sub-FSM if the operation is a write. The operation type is described by the Read Not Write (RnW) value.

In the case of a write operation, the *Flash Wait* sub-FSM redirects the main FSM based on the *FWait* counter value, which is incremented at each step to ensure transitions to the appropriate sub-FSM corresponding to the current phase of the write operation. First, the FSM is redirected to the *Erase* sub-FSM, which sets all logic cells in the current flash sector to ‘1’. Once this step is completed (as indicated by path D), control returns to the *Flash Wait* sub-FSM, which then redirects the FSM to the *Modify* sub-FSM. This sub-FSM utilizes the DMA, as shown by path B, and is responsible for updating the buffered sector data with the new data to be written back to flash. After completion (path E), the FSM transitions to the *Write* sub-FSM, which also uses the DMA, as illustrated by path C. Upon completion of this sub-FSM two cases are possible: all data has been written into flash then control returns once more to the *Flash Wait* sub-FSM, which finally redirects the FSM back to the *Idle* state, thereby completing the write operation. The FSM may need to be repeated in case the written data spans over multiple sectors which can be seen by the other potential case where one goes from the *Write* sub-FSM to the *Read* sub-FSM to repeat the previous steps on the next sector.

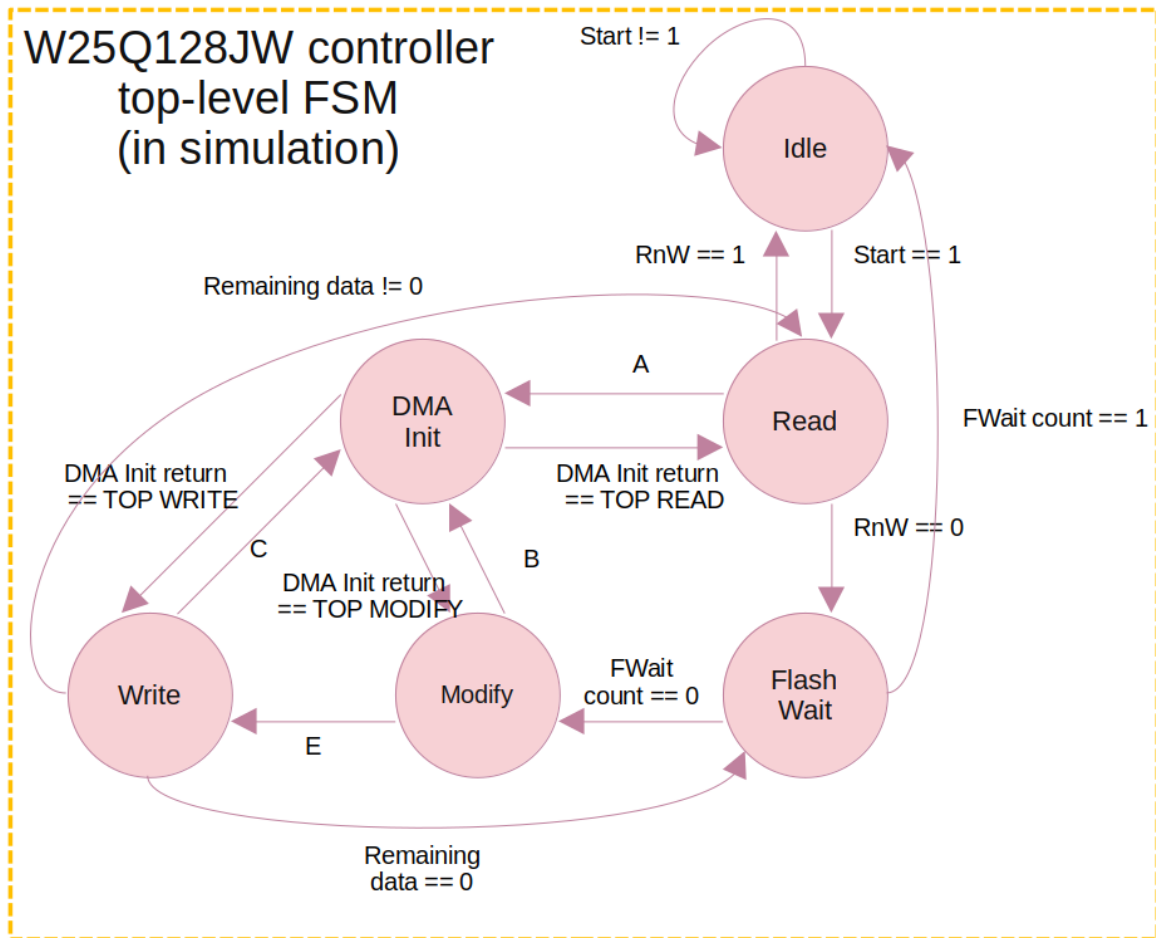


Figure 2: *W25Q128JW* controller top-level FSM architecture in simulation mode. Initial state is *Idle*.

In simulation mode, the top-level FSM is slightly modified. The *Flash Wait* sub-FSM is bypassed and performs an immediate redirection instead of actively waiting for the flash to indicate readiness for new commands (as described in the corresponding subsection below). In addition, the *Flash Wait* sub-FSM no longer redirects to the *Erase* sub-FSM, as this step would block the simulated FSM indefinitely. These steps are therefore omitted in simulation mode. This behavior is controlled using general compilation flags (`'ifndef FPGA.SYNTHESIS`

and `#ifndef SYNTHESIS`). The simulation-oriented implementation is illustrated in Figure 2.

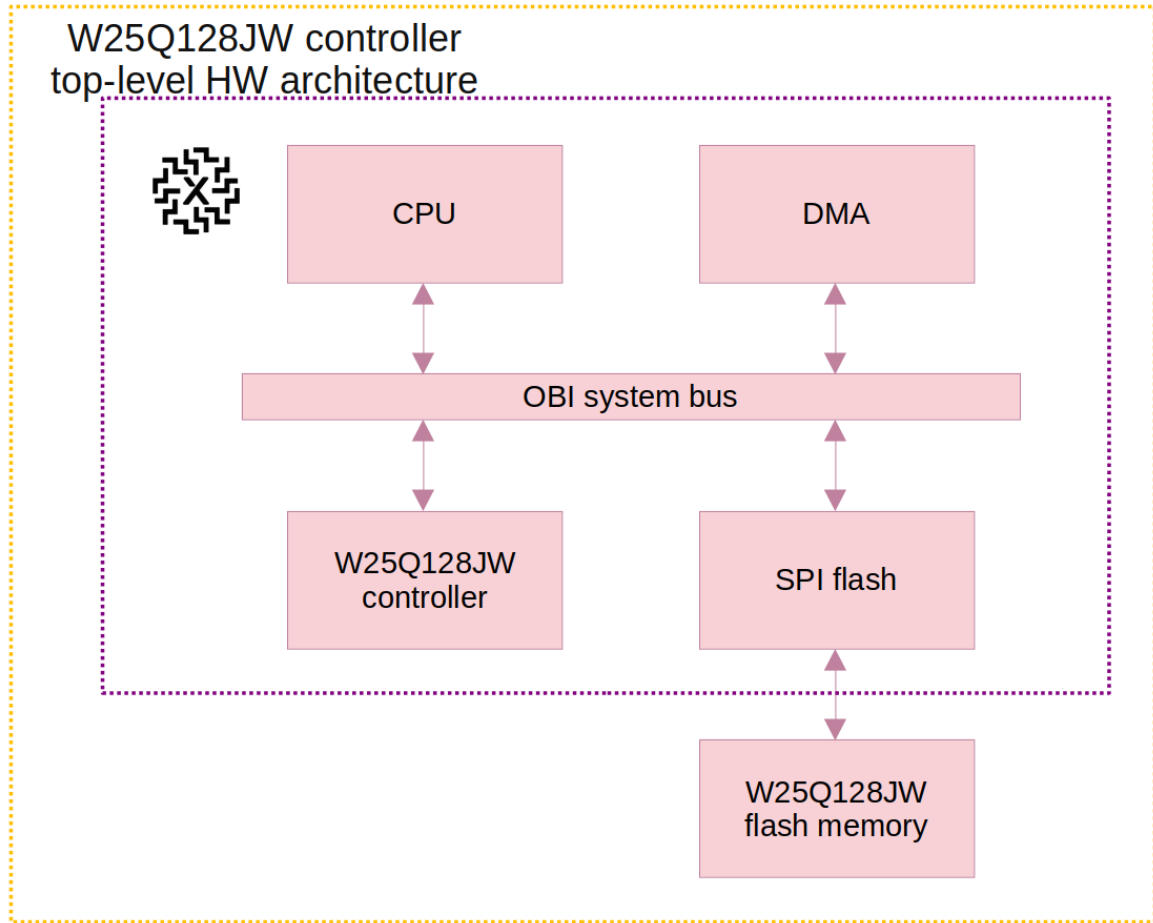


Figure 3: *W25Q128JW* controller top-level HW architecture.

The hardware architecture is based on an OBI system bus. Consequently, the W25Q128JW controller module incorporates an internal OBI FSM to communicate with the rest of the system components [4]. The controller interfaces with the CPU, the DMA, and the SPI flash controller, which in turn communicates with the W25Q128JW flash memory. The W25Q128JW flash memory is external to X-HEEP, whereas all other described components are integrated within the MCU.

The CPU configures the internal registers of the W25Q128JW controller (shown in Table 1) to specify the operation type (read or write), the target address, the operation byte length <sup>2</sup>, and to initiate the operation. In addition, the CPU can monitor the controller status through polling or interrupt based on the user’s needs.

The controller, in turn, communicates with the DMA during the *READ* sub-FSM to transfer data read from the flash temporarily stored in the SPI flash RX FIFO into the internal RAM. This data corresponds either to the number of bytes specified by the operation length in the case of a read operation, or to an entire sector in the case of a write operation.

During the *MODIFY* sub-FSM, the DMA is used to perform RAM-to-RAM data transfers. The data to be written to flash is copied into the sector buffer, where it is modified prior to being written back to flash.

<sup>2</sup>Read operation has byte-level granularity while the write operation has word-level granularity.

Finally, the DMA is exploited one last time during the *WRITE* sub-FSM to move data from the RAM sector buffer into the SPI flash TX FIFO. This last step is done page by page (256 bytes) for an entire sector (16 pages).

The second component which the W25Q128JW controller sets is the SPI flash controller. The module must issue multiple commands to the SPI flash to specify the requested operation, enable write access, and perform related control actions. These commands are described in more detail in the corresponding sub-FSM sections.

Table 1: W25Q128JW Controller Internal Registers

| Register Name | Bits | Field Name  | Description                                                                    |
|---------------|------|-------------|--------------------------------------------------------------------------------|
| CONTROL       | 1    | RNW         | Read (1) / Write (0) operation mode                                            |
|               | 0    | START       | Start operation trigger                                                        |
| STATUS        | 0    | READY       | Ready for new operation                                                        |
| F_ADDRESS     | 31:0 | F_ADDRESS   | (Flash_ADDRESS) Flash address to read from or write to                         |
| S_ADDRESS     | 31:0 | S_ADDRESS   | (Store_ADDRESS) RAM address to store data read from SPI flash                  |
| MD_ADDRESS    | 31:0 | MD_ADDRESS  | (ModifyData_ADDRESS) RAM address containing data used to modify flash contents |
| LENGTH        | 31:0 | LENGTH      | Number of bytes to read or write                                               |
| INTR_STATUS   | 0    | INTR_STATUS | Operation completion interrupt status                                          |

The module has been integrated into X-HEEP and is located at `hw/ip/w25q128jw_controller/`. The pull request with the module can be found at [5].

## 2.1 READ FSM

The details of the "READ" FSM are depicted on Figure 4.

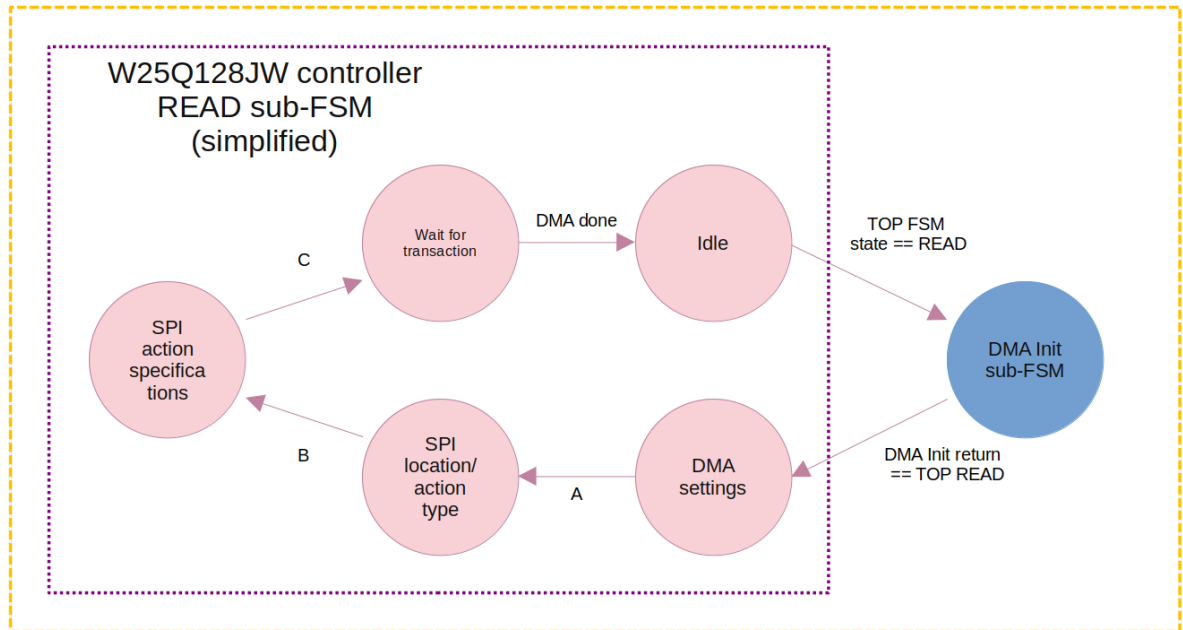


Figure 4: *W25Q128JW* controller READ FSM architecture. Initial state is *Idle*.

As with all other FSMs in the module, the initial state is *Idle*. When the top-level FSM enters the *READ* state, it activates the corresponding sub-FSM. As described previously, this sub-FSM first transitions to the *DMA Init* sub-FSM before any DMA operation is performed. Once control returns to the *READ* sub-FSM, the DMA is configured.

The DMA configuration performed in the *READ* sub-FSM defines the source and destination pointers, their increment behavior, the data types, the trigger source, and the transfer size. This configuration enables the transfer of data from the SPI RX FIFO to the RAM sector buffer.

Table 2: DMA configuration for READ operation

| Parameter      | DMA Source                              | DMA Destination               |
|----------------|-----------------------------------------|-------------------------------|
| Pointer        | SPI RX FIFO                             | RAM sector buffer (S_ADDRESS) |
| Increment      | 0 (FIFO head)                           | +4 (32-bit word)              |
| Data type      | 32-bit                                  | 32-bit                        |
| Trigger source | SPI RX FIFO to memory                   |                               |
| Transfer size  | Configured length in words (starts DMA) |                               |

The configured length is the user specified length in case of a read operation and is 4096 (sector is 4 KBytes) in case of a write operation.

Once this step is completed, the FSM transitions through path A to the next state, *SPI Location/Action Type*. This state represents a set of internal sub-states, grouped here for simplicity, as is the case for other states shown in this FSM. During this state, the SPI TX FIFO is monitored to ensure it is not full before issuing a new command. When space is available, if the SPI interface is ready, the command is written to the TX FIFO.

In the case of a read operation, the command consists of the specified target address combined with the flash read command code (FC\_RD). For a write operation, the address corresponds to the start address of the currently targeted sector, together with the flash read command code.

Following the previous state, the FSM takes path B to reach the *SPI Action Specification* state. The SPI interface is once again checked to ensure it is ready to accept additional commands. The command issued in this state specifies the direction, speed, and length of the read operation to be performed by the flash memory.

The direction field is set to ‘1’, indicating that data is transferred from the flash memory to the SPI RX FIFO. The speed field is set to ‘0’, specifying standard SPI operation. Finally, the transfer length, expressed in bytes, matches the value previously configured in the DMA transfer size (in bytes here).

Subsequently, the FSM transitions through path C to the *Wait for Transaction* state. In this state, the FSM waits for the DMA to assert its completion signal (only DMA channel 0 is used in this implementation) before redirecting the top-level FSM, as described previously, according to the type of operation being performed.

## 2.2 DMA\_INIT FSM

The *DMA Init* sub-FSM is responsible for resetting and initializing all DMA registers to a known, clean state prior to each DMA transfer. This step is required because the DMA peripheral retains its configuration between successive transfers, and any residual settings could lead to incorrect behavior. As a result, the *DMA Init* sub-FSM is invoked before every DMA usage described in this section. In addition to the mandatory register resets, optional DMA features such as zero padding and address mode configuration are also initialized when enabled. The



design of this sub-FSM is inspired by the reference DMA driver implementation provided in `sw/device/lib/drivers/dma/dma.c`.

### 2.3 FWAIT FSM

The FWAIT (Flash WAIT) sub-FSM is responsible for handling wait states associated with flash memory operations. During simulation, this sub-FSM primarily redirects control between the different sub-FSMs and emulates the waiting period during which the flash memory responds via the SPI interface to indicate completion of the issued command. The FWAIT sub-FSM is entered after the READ, ERASE, and WRITE FSMs.

The corresponding logic is shown in Figure 5.

Once started, the FSM enters the sequence that configures the RX watermark (RX WM) to a value of 1. This configuration ensures that the RX WM FIFO is asserted as soon as a single word is present in it, allowing the module to wait until data becomes available.

After the RX WM flag is risen, the FSM reads the word from the RX FIFO. The word to be retrieved is determined by the preceding command issued in the *SPI FC\_RSR1* state, which triggers a read of flash status register 1. This register contains the busy flag of the W25Q128JW flash memory. If the flash is still busy, the FSM loops back to the RX WM state and repeats the process. When the flash indicates readiness, the FSM returns to the *Idle* state and redirects the top-level FSM to the appropriate sub-FSM, as described previously.

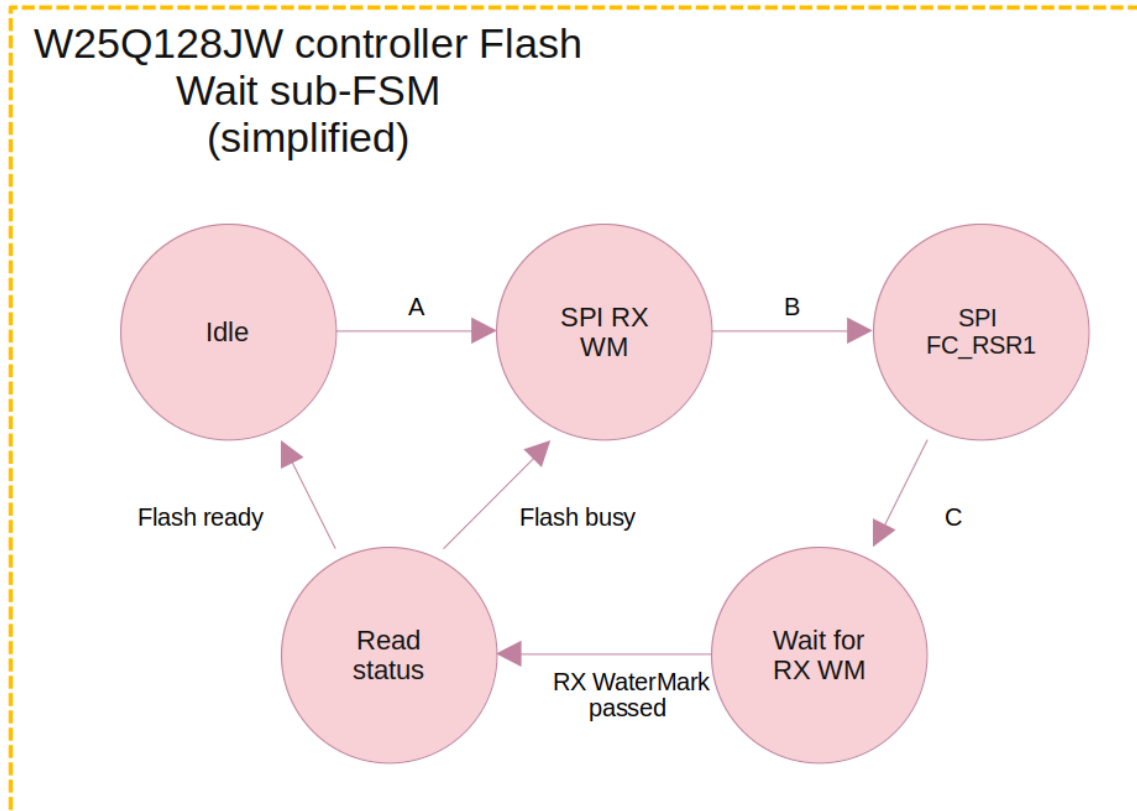


Figure 5: *W25Q128JW* controller FWAIT FSM architecture. Initial state is *Idle*.

## 2.4 ERASE FSM

The ERASE FSM configures the SPI interface with the appropriate target sector address and issues the flash sector erase command. In addition, it sends the flash write-enable command, which is required prior to any flash programming or erase operation. The corresponding logic is shown in Figure 6.

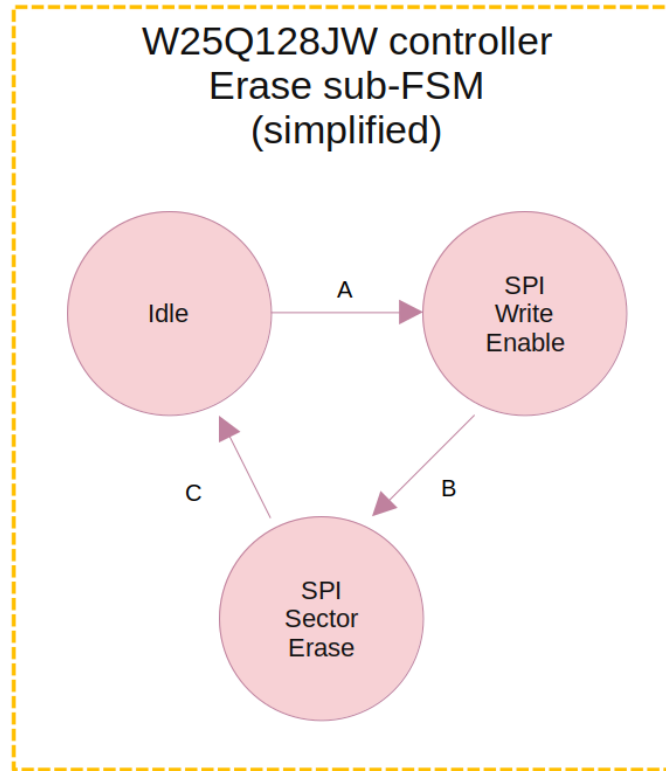


Figure 6: *W25Q128JW* controller ERASE FSM architecture. Initial state is *Idle*.

## 2.5 MODIFY FSM

Similar to the *READ* sub-FSM, the *MODIFY* sub-FSM is activated by the top-level FSM and immediately transfers control to the *DMA Init* sub-FSM. Once the DMA initialization is completed, the *MODIFY* sub-FSM proceeds with configuring the DMA to perform a memory-to-memory transfer. This transfer updates the sector buffer with the new data prior to writing it back to flash.

Table 3: DMA configuration for MODIFY operation

| Parameter      | DMA Source                              | DMA Destination               |
|----------------|-----------------------------------------|-------------------------------|
| Pointer        | RAM new data buffer (MD_ADDRESS)        | RAM sector buffer (S_ADDRESS) |
| Increment      | +4 (32-bit word)                        | +4 (32-bit word)              |
| Data type      | 32-bit                                  | 32-bit                        |
| Trigger source | Memory-to-memory                        |                               |
| Transfer size  | Configured length in words (starts DMA) |                               |

The source pointer must also account for the possibility of writing multiple sectors while the destination pointer must take into account the fact that the write operation may not begin at the start of the first sector. Any initial sector misalignment must therefore be considered to ensure that the right data is written at the right position.

The transfer size must account for similar considerations. The data to be written may be entirely contained within a single initial sector. However, any offset within the first sector, as well as the possibility that the write operation spans multiple sectors, must be taken into account. The amount of data successfully written within a sector is recorded, and the top-level FSM iterates as needed until all data has been written.

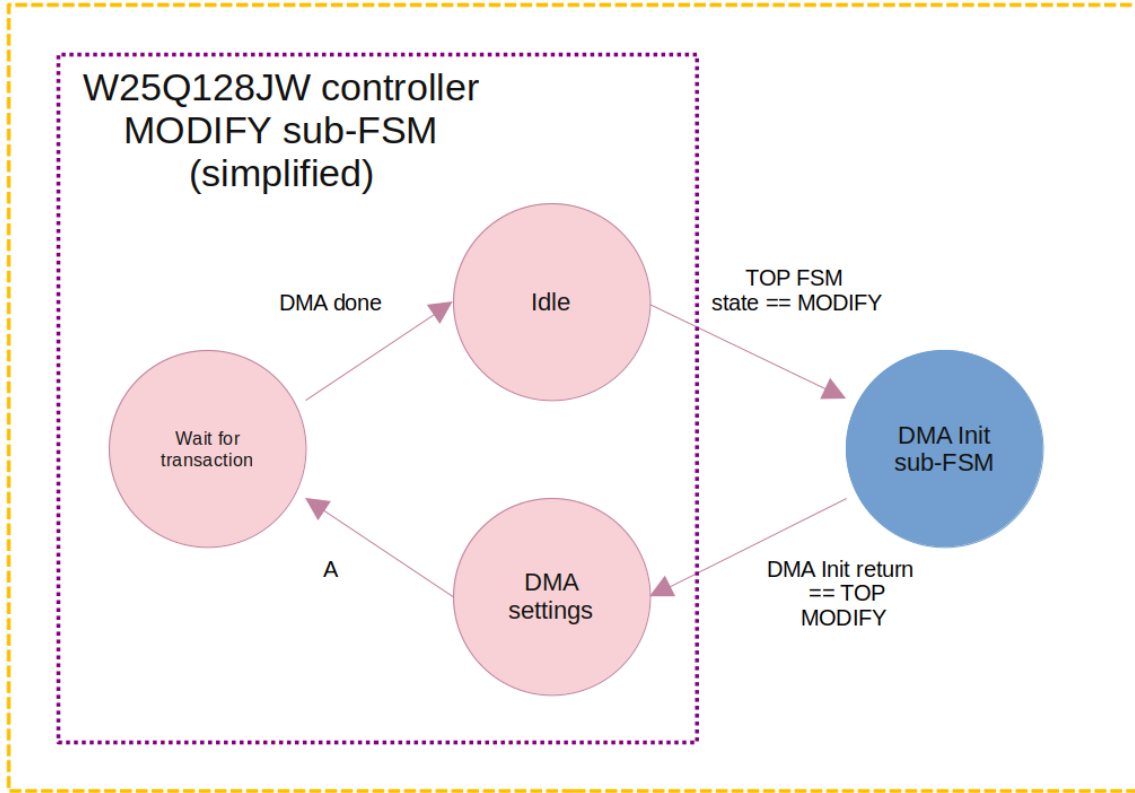


Figure 7: *W25Q128JW* controller MODIFY FSM architecture. Initial state is *Idle*.

## 2.6 WRITE FSM

The details of the "WRITE" FSM are depicted on Figure 8

The *WRITE* sub-FSM begins by following path A to a state that checks the SPI TX FIFO depth to ensure it can accept a new command. Once this condition is satisfied, the flash write-enable command (FC\_WE) is written to the SPI TX FIFO for processing by the SPI controller. Prior to issuing this command, the FSM verifies that the flash device is ready. The implementation of this sequence is inspired by the software reference located in `sw/device/bsp/w25q`.

Following this step, path B is taken to enter the page program command sequence. In this state, the FSM checks that sufficient space is available in the SPI TX FIFO before issuing a command. The command sent contains the flash page program instruction (FC\_PP) together with the start address of the currently targeted page within the current sector.

Once again, control is first transferred to the *DMA Init* sub-FSM before any DMA operation is performed. After initialization, the DMA is configured to transfer page data from the sector RAM buffer to the SPI TX FIFO in preparation for the flash write operation.

Once the DMA signals the completion of its transfer, the FSM enters the command sequence that instructs the SPI interface to program all data received in the TX FIFO into the flash

Table 4: DMA configuration for WRITE (page program) operation

| Parameter      | DMA Source                      | DMA Destination |
|----------------|---------------------------------|-----------------|
| Pointer        | RAM sector buffer + page offset | SPI TX FIFO     |
| Increment      | +4 (32-bit word)                | 0 (FIFO head)   |
| Data type      | 32-bit                          | 32-bit          |
| Trigger source | Memory to SPI TX FIFO           |                 |
| Transfer size  | Page word size: 64 words        |                 |

memory. If not all 16 pages have yet been written back, the FSM loops back to the *SPI Write Enable* state, as the write-enable command must be issued prior to each write or erase operation on the flash memory.

If all 16 pages have been successfully written back to flash and all data has been transferred, the write operation is complete. The FSM then transitions to the *Flash Wait* sub-FSM, which redirects the top-level FSM to the *Idle* state and signals the end of the operation. If this condition is not met, the top-level FSM loops back to the *READ* state to process the next sector, and this sequence repeats until all data has been written to flash.

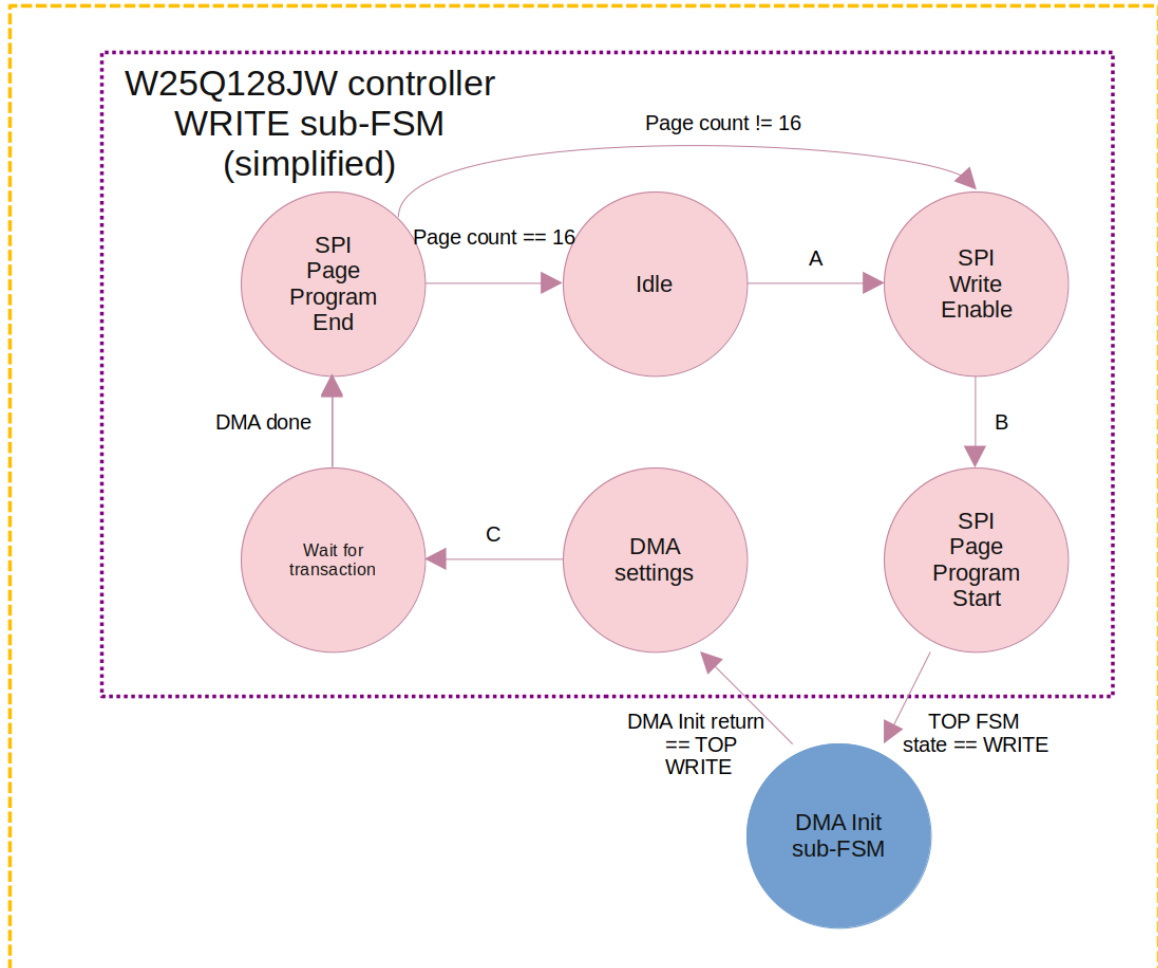


Figure 8: W25Q128JW controller WRITE FSM architecture. Initial state is *Idle*.

## 3 Software

The software component of this project consists of the W25Q128JW controller driver, which provides a software interface for configuring and operating the flash controller from the CPU. The driver implementation exposes functions to initiate read and write operations, configure operation parameters, and monitor operation completion using either polling or interrupt-based mechanisms. Furthermore, the software section includes three application examples that demonstrate proper usage of the driver and serve as proofs of concept for the project. These applications are described in the following subsections.

### 3.1 Driver

The W25Q128JW controller driver can be found at `sw/device/lib/drivers/w25q128jw_controller`.

#### 3.1.1 Operation

The driver allows the user to specify the operation type, the operation length in bytes, the flash address, the RAM buffer address, and the RAM address containing the data to be written (the latter being unused when a read operation is selected). These parameters are written to the controller's memory-mapped registers (see Table 1) before asserting the start signal, which completes the `w25q128jw_controller_rnw` operation.

#### 3.1.2 Polling

The polling completion mode is handled by the `w25q128jw_controller_is_ready_polling` function. The module's ready bit is read from the status register (see Table 1) and simply returned. This function can be looped and checked for the value '1' indicating the given operation is over.

#### 3.1.3 Interrupt

To provide a more elegant and energy-efficient mechanism for detecting operation completion, an interrupt-based approach is implemented. This method relies on a global completion flag, which is initially cleared before starting an operation. When the controller raises an interrupt, the corresponding IRQ handler sets this flag to indicate that the interrupt has been serviced. The driver then clears the internal interrupt status register of the controller (see Table 1). An example of such implementation can be found at `sw/applications/example_w25q128jw_read_interrupt`.

The interrupt handler is declared as `weak`, allowing users to override the default implementation and define custom interrupt-handling behavior through the `handler_irq_w25q128jw_controller` function.

To support this interrupt mechanism, modifications were required in the Platform-Level Interrupt Controller (PLIC) driver located in `sw/device/lib/drivers/rv_plic/rv_plic.c`. In addition, changes to the top-level X-HEEP design were necessary to integrate and properly route the W25Q128JW controller interrupt service [2].

### 3.2 Applications

The W25Q128JW controller applications can be found at `sw/applications` and are further detailed below. The main source of inspiration for such code comes from `sw/applications/example_spi_read` and `sw/applications/example_spi_write`.

### 3.2.1 Read example

The read examples located in `sw/applications/example_w25q128jw_read` and `sw/applications/example_w25q128jw_read_interrupt` consist of two main source files: a header file named `flash_data.h` and an application file `main.c`.

The `flash_data.h` file defines test data buffers, including a golden reference dataset stored in RAM for verification purposes, as well as a flash buffer containing identical data to be read by the controller.

The `main.c` file implements an example application that demonstrates reading data from the W25Q128JW flash memory and verifying that the retrieved data matches the golden reference. Two variants of the example are provided: one using polling-based completion detection, and another relying on interrupt-based completion detection.

**Polling** The polling-based version is straightforward and follows the sequence outlined below:

- Initialize the SPI flash interface.
- Launch the read operation using the W25Q128JW controller driver.
- Wait for operation completion by polling the controller status.
- Verify the read data against the expected reference values.

**Interrupt** The interrupt-based version follows a similar pattern but additionally activates the W25Q128JW controller module interrupt in PLIC as well as global interrupts.

### 3.2.2 Write example

The write example application consists of two main source files: `ram_new_data.h` and `main.c`. The `ram_new_data.h` file defines the RAM test data buffer containing the data to be written to flash memory during write operations.

The `main.c` file implements an example application that demonstrates writing data to the W25Q128JW flash memory, followed by a read-back operation to verify data integrity. The test configuration uses a transfer size of 4100 bytes, spanning two flash sectors (independent of sector misalignment), with write operations performed at word-level precision.

The sequence is outlined below:

- Initialize the SPI flash interface.
- Launch the write operation using the W25Q128JW controller driver.
- Wait for write completion through polling.
- Launch a read operation to retrieve the written data.
- Wait for read completion through polling.
- Verify the read-back data against the original input data.

## 4 Testing

To verify the module implementation, the Siemens QuestaSim tool was used [6]. QuestaSim provides a comprehensive simulation and debugging environment, enabling functional verification and efficient debugging of the W25Q128JW controller within the X-HEEP platform.

This example (described in the above **Applications** sub-section) is well suited to illustrate the different waveform sections observed in QuestaSim, as it includes both a write operation and a subsequent read operation used to verify the correctness of the written data.

All signals are grouped under the following: `//  
tb_top/testharness_i/x_leep_system_i/core_v_mini_mcu_i/.`

The DMA signals are located deeper in the design hierarchy at:  
`ao_peripheral_subsystem_i/dma_subsystem_i/dma_i_gen[0]/dma_i.`

The SPI flash controller signals are located deeper in the design hierarchy at: `ao_peripheral_subsystem.i/spi_subsystem.i/ot_spi.i`.

The W25Q128JW controller signals are located deeper in the design hierarchy at: `peripheral_subsystem_i/w25q128jw_controller_i`.



Figure 9: QuestaSim overall waveform view of X-HEEP executing the `example_w25q128jw_write` application.

Figure 9 presents signals from the SPI flash controller, the DMA engine, and the W25Q128JW controller. The origin of each signal group is indicated by the colored band on the left-hand side of the figure: red for the SPI flash controller, green for the DMA engine, and blue for the W25Q128JW controller.

The waveforms initially show the program and data transfer from the W25Q128JW flash memory to the internal X-HEEP RAM, highlighted by the purple section. This behavior occurs because the application is built using the `LINKER=flash_load` option, which requires the program to be loaded from flash. Consequently, the simulation is executed with `boot_sel=1` using the command `make run-gui RUN_OPT=1 PLUSARGS="c firmware=../../sw/build/main.hex boot_sel=1 execute_from_flash=0`, triggering the initial flash-to-RAM loading sequence. As observed in the waveforms, this section heavily involves the SPI controller. The write (orange section) and read (blue section) sequences are described in more detail below.

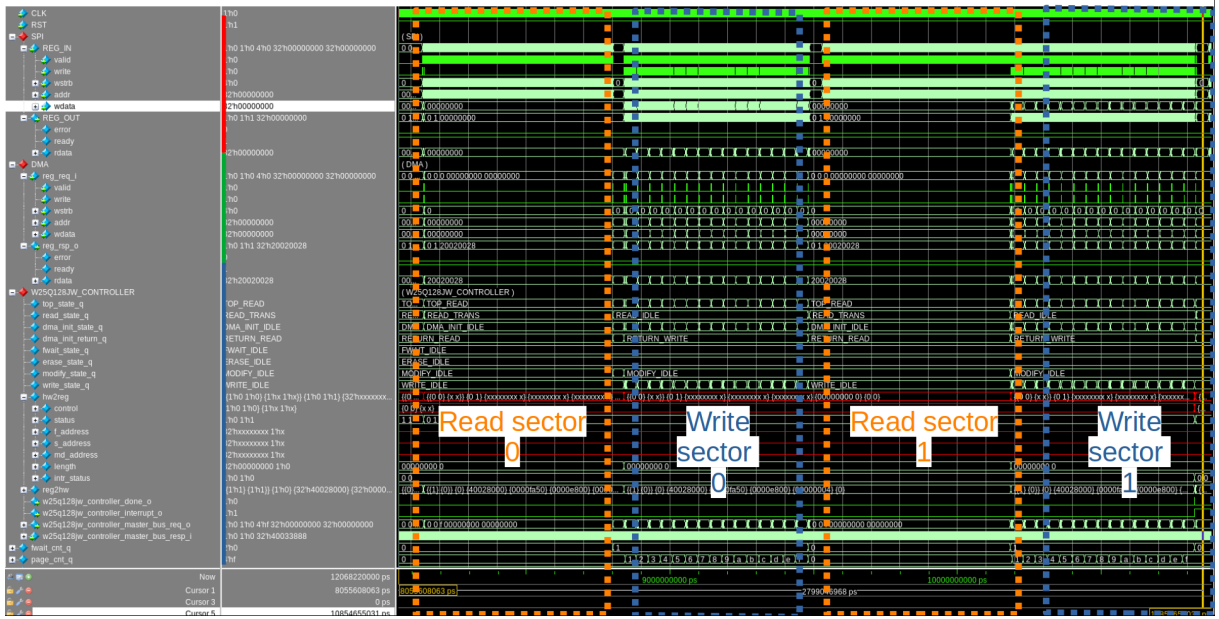


Figure 10: QuestaSim write section waveform view of X-HEEP executing the write section of the example.w25q128jw\_write application.

Figure 10 provides a zoomed view of the orange-highlighted section in Figure 9. In our example, the write operation transfers 4100 bytes, spanning two flash sectors, and therefore iterates over the top-level FSM shown in Figure 2. Since the design is simulated, the erase step is omitted and the *FWAIT* sub-FSM only redirects control to the appropriate sub-FSM.

The read sequences begin with a DMA initialization followed by DMA reconfiguration, after which the SPI interface is programmed and the FSM waits for completion. This process results in the target sector being read and its contents stored in the RAM sector buffer, as described in Figure 4.

Between the orange and blue sections, the *MODIFY* sub-FSM is executed (see Figure 7), followed by the *WRITE* sequence (see Figure 8) in blue. During this phase, the write operation iterates over the 16 pages of the sector, each requiring a DMA initialization and configuration, which explains the repeated DMA signal activity observed in the waveforms (16 peaks in total).

Since the entire data set does not fit within a single sector, the same process is repeated for the second sector until all data has been written to the flash memory.



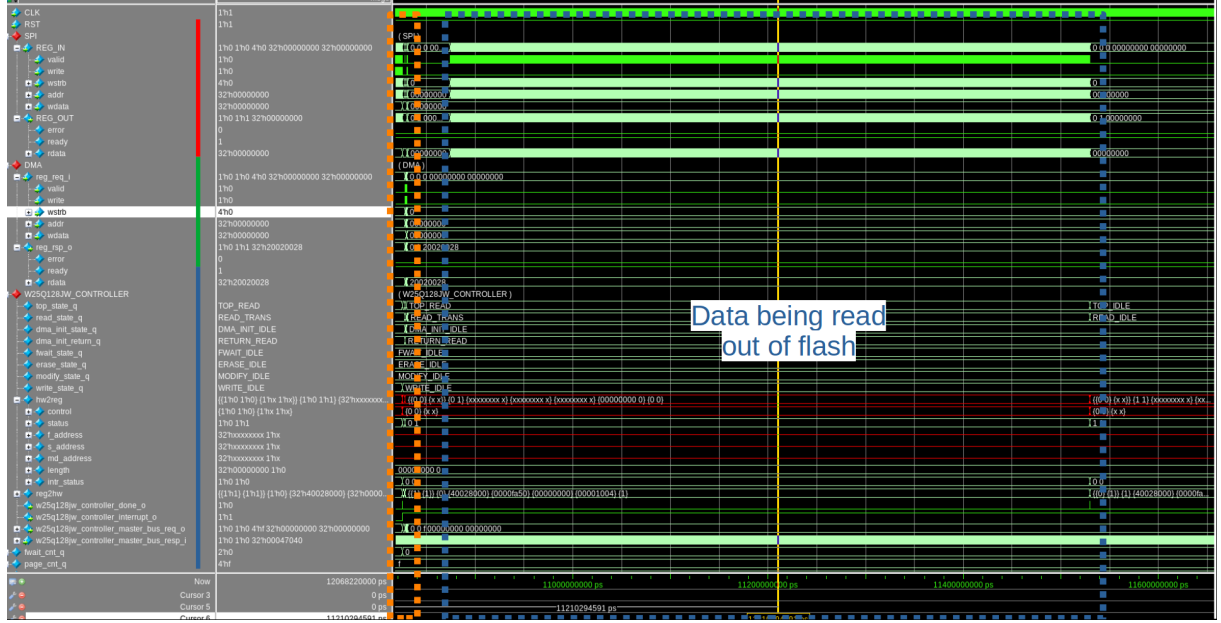


Figure 11: QuestaSim read section waveform view of X-HEEP executing the read section of the example.w25q128jw\_write application.

Figure 11 provides a zoomed view of the blue highlighted section in Figure 9. In this case, a single read sequence is sufficient to retrieve all 4100 bytes for verification. This is because, during read operations, the SPI transfer length configured for the FC\_RD command directly corresponds to the user-specified data length in the *READ* sub-FSM, rather than being constrained to a full sector size as in write operations.

The operation starts with the orange section showcasing DMA initialization and programming which are followed by SPI programming. Following completion of the read operation, the internal status register of the W25Q128JW controller is updated to indicate that the module is no longer busy and is ready to accept a new operation. This behavior can also be seen at the end of Figure 10.

## 5 Conclusion

Throughout this project, a dedicated hardware controller for the W25Q128JW serial NOR flash memory was designed, implemented, and integrated into the X-HEEP platform. The proposed controller offloads complex and time-critical flash management tasks from the CPU by autonomously orchestrating SPI transactions and DMA transfers through a hierarchical FSM architecture. Both read and write operations are supported, including sector-based erase as well as handling alignment constraints and multi-sector transfers.

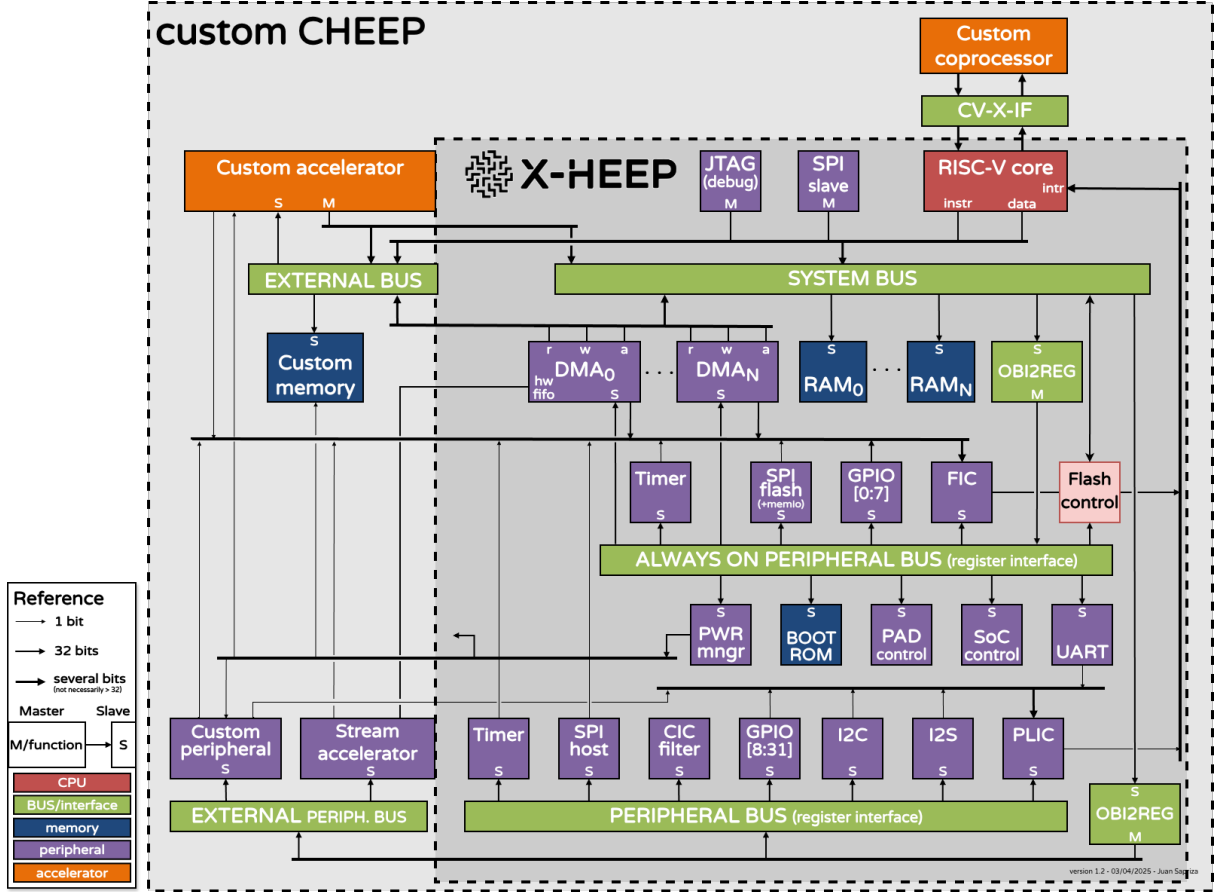


Figure 12: X-HEEP architecture including the W25Q128JW controller in pink. Diagram originates from [2].

The controller was verified through simulation using software composed of a dedicated driver and multiple application examples. The software interface enables both polling-based and interrupt-driven operation completion detection, with the latter providing a more energy-efficient execution model. Necessary modifications to the PLIC and X-HEEP top-level integration were performed to support the new peripheral and its interrupt service.

As a continuation of this project, planned for the Spring semester 2026, several potential directions for further development have been identified. One promising improvement consists in enabling direct communication between the W25Q128JW controller and the SPI and DMA peripherals, bypassing the system bus. Since the system bus is heavily utilized during flash operations, directly configuring the SPI and DMA internal registers could significantly reduce execution time by allowing all configuration parameters to be set within a single cycle, rather than incurring the overhead of sequential bus transactions. This approach could also lead to reduced power consumption.

In addition, the W25Q128JW controller could be extended to provide greater configurability by allowing the user to select the SPI operating mode, such as single, dual, or quad SPI. Finally, minor enhancements could be introduced to support byte-level granularity for write operations, should such functionality be required by future applications.

The hardware source files associated with the W25Q128JW controller, excluding files modified for integration into X-HEEP, are located in `hw/ip/w25q128jw_controller`. Similarly, the software source files, excluding those modified for integration into X-HEEP, are located in `sw/applications` and `sw/device/lib/drivers/w25q128jw_controller`.

I would like to sincerely thank my supervisors, Dr. Pasquale Davide Schiavone and PhD Candidate Tommaso Terzano, for their continuous support and guidance throughout the Fall semester. A great deal was learned through their insightful and constructive mentorship.

I would also like to express my gratitude to the remaining staff of the Embedded Systems Laboratory at EPFL for their support during this project, as well as to Prof. David Atienza Alonso for this opportunity.

The W25Q128JW controller is to be integrated into X-HEEP via a dedicated pull request to the official repository [5].

## References

- [1] OpenTitan Project. “Spi host ip — opentitan hardware documentation,” Accessed: Jan. 6, 2026. [Online]. Available: [https://opentitan.org/book/hw/ip/spi\\_host/](https://opentitan.org/book/hw/ip/spi_host/).
- [2] École Polytechnique Fédérale de Lausanne (EPFL). “X-heap — extended hardware environment for embedded processors,” Accessed: Jan. 6, 2026. [Online]. Available: <https://x-heap.epfl.ch/>.
- [3] Winbond Electronics. “W25q128jw serial nor flash memory,” Accessed: Jan. 6, 2026. [Online]. Available: [https://www.winbond.com/hq/product/code-storage-flash-memory/serial-nor-flash/?\\_locale=en&partNo=W25Q128JW](https://www.winbond.com/hq/product/code-storage-flash-memory/serial-nor-flash/?_locale=en&partNo=W25Q128JW).
- [4] OpenHW Group. “Obi system bus specification v1.6.0,” Accessed: Jan. 6, 2026. [Online]. Available: <https://github.com/openhwgroup/obi/tree/072d9173c1f2d79471d6f2a10eae59ee387d4c6f>.
- [5] Thomas Lenges. “Integration of the w25q128jw controller into x-heap.” Pull request. [Online]. Available: <https://github.com/x-heap/x-heap/pull/795>.
- [6] Siemens Digital Industries Software. “Questasim verification platform,” Accessed: Jan. 7, 2026. [Online]. Available: <https://static.sw.cdn.siemens.com/siemens-disw-assets/public/QzFgMxW5gizEDRIAzyTQE/fr-FR/Siemens-SW-Questasim-FS-85329-D5.pdf>.